

프레임워크 개발자 가이드

CRUD 부터 Transaction·오류·Domain 호출·Messaging 까지 실제 개발 결정을 빠르게 내리는 실무 참조서

CPF 업무 개발자·리드 개발자를 위한 실무 참조서입니다. API 를 선택하고 옵션을 설정해 구현·실패 처리·검증까지 끝냅니다.

- 하려는 일을 기준으로 API 를 찾습니다.
- 옵션의 실제 기본값과 변경 조건을 확인합니다.
- 실패·UNKNOWN·복구까지 구현한 뒤 검증 명령으로 닫습니다.

목차

필요한 작업을 먼저 찾고 해당 장으로 이동합니다. 페이지 번호는 최종 PDF 기준입니다.

1 개발 시작: 무엇을 만들지 먼저 고르기.....	4
업무 니즈별 시작점.....	4
개발 중 검증 3 단계.....	4
2 CRUD 와 Persistence.....	5
Public API 빠른 선택.....	5
옵션 빠른 확인.....	6
JDBC · MyBatis · JPA 선택.....	6
최소 Transaction 예.....	7
3 Transaction · Idempotency · UNKNOWN.....	7

Public API 빠른 선택.....	7
옵션 빠른 확인.....	8
거래 패턴 선택.....	9
4 Domain 호출과 외부 연계.....	9
Public API 빠른 선택.....	10
옵션 빠른 확인.....	10
토폴로지별 호출.....	11
Canonical System6.....	11
5 오류 처리와 결과 분기.....	12
오류 상황별 개발자 결정.....	12
Public API 빠른 선택.....	12
결과 4 상태를 명시적으로 달는 예.....	13
6 Messaging · Cache · File · 업무 공통.....	13
Public API 빠른 선택.....	13
옵션 빠른 확인.....	14
7 Security · 승인 · 감사 · Logging.....	14
Public API 빠른 선택.....	15
옵션 빠른 확인.....	15
8 Starter · Generator · OpenAPI 연결.....	16

- Public Starter 빠른 선택.....16
- Generator 작업 순서..... 17
- 9 검증과 완료 판단..... 17
 - 검증 단계..... 17
- 10 고객 공통 Library 와 개발 환경..... 18
 - 고객 공통 JAR 생성·선택 연결..... 18
 - Windows 경로와 Docker Lifecycle..... 18
 - Source·EDU 길찾기..... 18

1 개발 시작: 무엇을 만들지 먼저 고르기

새 업무 API 를 만들 때 Internal 구조를 외우지 않고 Profile·Capability·검증 경로를 고릅니다.

업무 니즈별 시작점

표 1-1. 업무 니즈별 시작점

업무 니즈별 시작점의 선택 기준과 확인 항목을 빠르게 비교합니다.

하려는 일	먼저 선택	추가 Capability	첫 검증
일반 REST 업무 API	cpf-starter-web-api	JDBC/MyBatis/JPA, Common 등 필요한 것만	./gradlew cpfVerifyFast

하려는 일	먼저 선택	추가 Capability	첫 검증
인증·인가 API	cpf-starter-secure-api	OIDC/Session Provider	cpfVerifyTargeted(security)
UI 용 BFF	cpf-starter-bff	Security/Common/Integration	BFF + OpenAPI consumer
이벤트 중심 서비스	cpf-starter-event	Kafka/JMS/RabbitMQ/IBM MQ 중 선택	messaging targeted
Batch	cpf-starter-batch	DB/메시징/파일 필요 Capability	Batch developer guide

권장 흐름 Profile 하나를 시작점으로 선택하고 Provider/Capability 는 필요한 것만 추가합니다. Internal leaf Starter 를 업무 코드가 직접 의존하지 않습니다.

개발 중 검증 3 단계

```
./gradlew cpfVerifyFast
./gradlew cpfVerifyTargeted -PcpfTargetCapabilities=cache,messaging
./gradlew cpfVerifyFullLocal
```

- Generator 는 create/setup --preview/sync/diff/remove lifecycle 을 사용합니다.
- Generated-owned 파일 사용자 수정이 있으면 fail-closed 하고 unmanaged/custom Source 를 임의 삭제하지 않습니다.
- Generated Domain 의 online 은 필수이고 batch 는 선택입니다.

2 CRUD 와 Persistence

조회·등록·수정·삭제를 만들 때 어떤 CPF 계약과 Provider 를 써야 하는지 바로 선택합니다.

Public API 빠른 선택

표 2-1. Public API 빠른 선택

Public API 빠른 선택의 선택 기준과 확인 항목을 빠르게 비교합니다.

하려는 일	Public API / Annotation	핵심 옵션	언제 선택	실패·복구
HTTP 거래 진입	@CpfController	value=""	업무 Controller 의 표준 진입점	표준 Web/Error/Context 계약을 우회하지 않음
업무 규칙	@CpfService	value=""	업무 Service 경계	Controller 에 업무 규칙 집중 금지
Repository 선언	@CpfRepository	value=""	Persistence Adapter/Repository 경계	Owner DB 외 직접 조회 금지
단건 CRUD	CpfCrudRepository<T,ID>	save/findById/existsById/count/deleteById	표준 CRUD	findById Optional 결과를 업무 의미로 변환
목록 조회	업무 Repository 검색 메서드 + @CpfRepository	criteria + limit + CpfSort allow-list	코드/콤보/소량 결과처럼 최대 건수가 업무적으로 제한된 목록	무제한 List 조회 금지, 빈 목록은 정상 결과
Offset Page / Slice	CpfPagingAndSortingRepository<T,Q>	findAll/findSlice + CpfPageRequest + CpfSort	페이지 번호·전체 건수 필요 시 Page, count 가 불필요하면 Slice	0-based page, size<=200, deep offset·count 비용 확인
Keyset / Cursor	CpfCursorRepository<T,Q> + CpfCursorCodec	findAfter + CpfCursorPage + stable sort	대량·실시간 이력의 다음 페이지 탐색	signed cursor 검증, 안정 정렬키·tie-break 필수
Bulk	CpfBulkRepositoryPort	insertAll/updateAll/deleteAllById	다건 쓰기	batch size·Tx 범위·메모리·재실행 검토
Lock 조회	CpfLockingRepositoryPort	CpfLockMode	동시 수정 충돌 제어	장시간 PESSIMISTIC lock 금지
Local Transaction	@CpfTransactional	propagation/isolation/timeout/readOnly/rollback	한 DB Local Transaction	Remote Side Effect 를 물리 Tx 로 간주하지 않음

옵션 빠른 확인

표 2-2. 옵션 빠른 확인

옵션 빠른 확인의 선택 기준과 확인 항목을 빠르게 비교합니다.

API	옵션	필수/기본값	언제 변경	주의
CpfPageRequest	page	0-based	다음/이전 페이지	음수 금지
CpfPageRequest	size	기본 20, 최대 200	화면/Batch 요구에 맞춤	무제한 조회 금지
CpfPagingAndSortingRepository	findAll / findSlice	Page 는 count 포함, Slice 는 count 생략 가능	전체 건수/페이지 점프 필요 여부에 따라 선택	정렬 allow-list 와 stable ordering 을 Repository 에서 강제
CpfCursorRepository	findAfter	size 1~200 + 안정 정렬	대량·실시간 다음 페이지	임의 page jump 대신 next cursor 사용
CpfCursorCodec	signed cursor	외부 API Cursor 는 서명 검증	Cursor 를 Client 에 노출할 때	위변조 토큰 거부, raw PK/단순 Base64 직접 노출 금지
CpfLockMode	mode	NONE	낙관/비관 잠금 필요 시	OPTIMISTIC, OPTIMISTIC_FORCE_INCREMENT, PESSIMISTIC_READ, PESSIMISTIC_WRITE
@CpfTransactional	propagation	REQUIRED	호출 Tx 결합 정책 변경 시	Spring 의미 그대로, 임의 복제 금지
@CpfTransactional	isolation	DEFAULT	DB 격리 수준을 명시해야 할 때	DB3 실제 동작 차이 검증
@CpfTransactional	timeout	TIMEOUT_DEFAULT	긴 거래 차단	timeoutString 과 중복 정책 주의

API	옵션	필수/기본값	언제 변경	주의
@CpfTransactional	readOnly	false	조회 전용 Tx	쓰기 보장으로 오해 금지
@CpfTransactional	rollbackFor/ noRollbackFor	빈 배열	표준 rollback 규칙 보정 시	업무 실패와 기술 실패 의미를 먼저 정의

JDBC · MyBatis · JPA 선택

표 2-3. JDBC · MyBatis · JPA 선택

JDBC · MyBatis · JPA 선택의 선택 기준과 확인 항목을 빠르게 비교합니다.

선택	적합한 경우	피해야 하는 경우	개발 시 확인
JDBC	SQL을 직접 제어하고 단순 Mapping이 많은 업무	복잡 객체 그래프 관리가 중심	SQL, index, lock, vendor 차이
MyBatis	명시적 SQL/Mapper가 팀 표준인 업무	단순 CRUD에 Mapper만 과도하게 늘어나는 경우	Mapper SQL + DB3
JPA	Entity lifecycle/ORM이 이점인 업무	복잡 대량 SQL·Vendor 특화 튜닝이 중심	fetch/flush/lock/N+1 + DB3

- 공통 완료조건 — Controller → Service → Repository → Owner DB 경로로 실제 조회/변경을 실행하고 Transaction rollback, 오류 mapping, DB 결과, 재실행 결과를 Test로 확인합니다.
- JPA — Service는 업무 Repository만 호출합니다. Repository 내부에서 JpaRepository/EntityManager를 사용하고 fetch/flush/lock/N+1과 Vendor별 DDL·Query 차이를 확인합니다.
- MyBatis — Service는 업무 Repository만 호출하고 Repository가 `@Mapper`를 사용합니다. Mapper/SQL은 Repository 내부 Persistence 구현이며 Parameter/Result mapping, Paging, Transaction, DB3 SQL 차이를 함께 검증합니다.
- JDBC — Service는 업무 Repository만 호출하고 Repository 내부에서 `JdbcTemplate`/`NamedParameterJdbcTemplate`을 사용합니다. SQL·Index·Lock·Vendor 차이를 Oracle/PostgreSQL/MariaDB에서 확인합니다.

선택표는 시작점입니다. 실제 업무 호출은 Controller → Service → Repository로 고정하고, Mapper/JdbcTemplate/EntityManager는 Repository 내부 구현으로 둔 뒤 Transaction과 DB3 검증까지 연결합니다.

Controller · Service · Repository 조회 기본 흐름

조회 기능은 단건 → 제한된 목록 → Offset Page/Slice → Keyset Cursor 네 패턴으로 시작하면 됩니다. HTTP 요청·응답은 Controller, 조회 조건과 read-only Transaction 은 Service, 실제 Owner DB 접근은 Repository 가 소유합니다. MyBatis Mapper·JdbcTemplate·EntityManager 는 Repository 내부 구현이므로 Service 나 Controller 에서 직접 호출하지 않습니다.

Golden Path HTTP 요청 → Controller → Service → Repository → Owner DB → Response DTO. 개발자는 먼저 이 호출 경로를 만든 뒤 조회 방식에 따라 Page/Keyset 옵션을 선택합니다.

표 2-4. 조회 방식 선택

화면 요구와 데이터 규모를 기준으로 단건·목록·Offset·Keyset 중 하나를 선택합니다. 실제 Generated Sample 은 단건·Offset·Keyset 을 Controller → Service → Repository 로 구현하고, 작은 비페이징 목록은 업무상 최대 건수가 명확할 때만 같은 경계로 별도 Endpoint 를 둡니다. Offset 응답은 SamplePage(items,total,page,size), Keyset 응답은 SampleSlice(items,hasNext,nextCursor) 형태입니다.

조회 방식	Controller 입력·반환	Service → Repository	선택 기준
단건 조회	GET /{id} → T 또는 404	detail(id) → findById(id)	상세 화면·PK/업무키 1 건
목록 조회	GET /list?... → List<T>	list(criteria) → bounded search	코드/선택 목록처럼 최대 건수가 작고 명확할 때
Offset Page / Slice	page,size,sort → CpfPage<T>/CpfSlice<T>	findAll/findSlice(criteria,page,sorts)	페이지 번호·전체 건수 필요 시 Page, count 불필요 시 Slice
Keyset / Cursor	cursor,size → CpfCursorPage<T>	findAfter(criteria,after,size,sorts)	대량·실시간 next 탐색, deep offset 회피

단건 조회 — 상세 화면에서 가장 먼저 만드는 기본 API

```
// Controller: GET /api/v1/member/samples/{id} → 200 SampleItem / 표준 404
@GetMapping("/{id}")
public ResponseEntity<SampleItem> detail(@PathVariable long id) {
    return ok(service.detail(id));
}

// Service: read-only Transaction + 업무 의미 변환
@CpfTransactional(transactionManager="cpfDomainTransactionManager", readOnly=true)
public SampleItem detail(long id) {
    return repository.findById(id).orElseThrow() ->
        new CpfBusinessException(CpfErrorCode.NOT_FOUND, "Sample 을 찾을 수 없습니다.");
}
```

Controller 는 ID 를 받고 Service 결과를 그대로 표준 응답으로 반환합니다. Repository 가 빈 값을 반환하면 Service 에서 NOT_FOUND 같은 업무/API 의미로 변환합니다. Controller 가 Repository 나 Mapper 를 직접 호출하지 않습니다.

목록 조회 — 결과 상한이 업무적으로 보장되는 작은 목록에만 사용 (패턴 예제)

```
// 업무상 최대 100 건처럼 상한이 명확할 때만 별도 목록 Endpoint 를 둡니다.
@GetMapping("/list")
public ResponseEntity<List<SampleItem>> list(@RequestParam String statusCode) {
    return ok(service.list(statusCode));
}

// Service -> Repository. limit 없는 전체 조회 금지
public List<SampleItem> list(String statusCode) {
    return repository.search(null, statusCode, 0, 100, "sample_item_id", "ASC");
}
```

이 `/list`는 조회 패턴 설명용 예시입니다. 현재 Generated Sample 의 공식 실행 예는 `/api/v1/member/samples` Offset 와 `/slice` Keyset 입니다. 비페이징 List 는 코드·콤보·소량 기준정보처럼 결과 상한이 확실한 경우만 사용하고, 커질 가능성이 있으면 처음부터 Offset 또는 Keyset 을 사용합니다.

Offset Page / Slice — 일반 검색 화면과 관리자 목록의 기본 선택

```
// Controller: 실제 Generated Sample 의 Offset 조회
@GetMapping
public ResponseEntity<SamplePage> search(
    @RequestParam(required=false) String keyword,
    @RequestParam(defaultValue="0") Integer page,
    @RequestParam(defaultValue="20") Integer size) {
    return ok(service.search(new SampleSearchRequest(keyword, null, page, size, null)));
}

// Service: Repository 목록 + count -> SamplePage(items,total,page,size)
public SamplePage search(SampleSearchRequest r) {
    int page=r.safePage(), size=r.safeSize();
    return new SamplePage(repository.search(r.keyword(), r.statusCode(), page, size,
        "sample_item_id", "ASC"), repository.count(r.keyword(), r.statusCode(), page, size);
}
```

`CpfPageRequest` 공통 계약은 0-based page, 기본 size 20, 최대 200 입니다. 전체 건수와 임의 페이지 이동이 필요하면 Page 를 사용하고, count 비용이 불필요하면 Slice 를 선택 합니다. Repository 가 offset 계산-정렬 allow-list 를 책임지고 Oracle/PostgreSQL/MariaDB 에서 같은 정렬 의미를 유지합니다.

Keyset / Cursor — 대량 이력·실시간 목록에서 다음 페이지를 빠르게 탐색

```
// Controller: 실제 Generated Sample 의 Keyset 조회
@GetMapping("/slice")
public ResponseEntity<SampleSlice> slice(
    @RequestParam(defaultValue="0") Long cursor,
    @RequestParam(defaultValue="20") Integer size) {
    return ok(service.slice(new SampleSearchRequest(null, null, null, size, cursor)));
}

// Service: size+1 조회 -> SampleSlice(items,hasNext,nextCursor)
public SampleSlice slice(SampleSearchRequest r) {
    int size=r.safeSize();
    List<SampleItem> rows=repository.cursorSlice(null, null, r.safeCursor(), size+1);
    boolean hasNext=rows.size()>size;
    List<SampleItem> items=hasNext ? List.copyOf(rows.subList(0,size)) : List.copyOf(rows);
    Long nextCursor=hasNext && !items.isEmpty() ? items.get(items.size()-1).getSampleItemId() : null;
    return new SampleSlice(items, hasNext, nextCursor);
}
```

Generated Sample 은 흐름을 쉽게 확인하도록 numeric keyset cursor 를 사용합니다. 실제 외부 Client 에 Cursor 를 노출하는 Public API 는 `CpfCursorRepository`와 `CpfCursorCodec`(HMAC 구현 제공)으로 서명·위변조를 검증합니다. Keyset 은 `createdAt DESC, id DESC`처럼 동물을 깨는 stable unique tie-break 를 포함해야 하며, 임의 페이지 점프가 중요하면 Offset Page 를 선택합니다.

Repository 구현 — DB 접근은 이 계층에서만 수행합니다.

@CpfRepository

```
public class SampleTransactionRepository extends MemberBaseRepository {
    private static final Set<String> SORT_COLUMNS = Set.of(
        "sample_item_id", "sample_key", "item_name", "status_code", "created_at");
    private final SampleTransactionMapper mapper;

    public Optional<SampleItem> findById(long id) {
        return Optional.ofNullable(mapper.findById(id));
    }

    public List<SampleItem> search(String keyword,String status,int page,int size,
        String sortBy,String direction) {
        int normalizedSize=pageSize(size);
        String column=SORT_COLUMNS.contains(sortBy)?sortBy:"sample_item_id";
        String order="DESC".equalsIgnoreCase(direction)?"DESC":"ASC";
        return mapper.search(keyword,status,pageOffset(page,normalizedSize),normalizedSize,column,order);
    }

    public List<SampleItem> cursorSlice(String keyword,String status,long cursor,int size) {
        return mapper.cursorSlice(keyword,status,cursor,boundedSize(size,21,201));
    }
}
```

MyBatis 를 선택한 경우 Repository 가 Mapper 를 호출합니다. JDBC 는 Repository 내부에서 JdbcTemplate/NamedParameterJdbcTemplate 을, JPA 는 Repository 내부에서 JpaRepository/EntityManager 를 사용합니다. 어떤 구현을 선택해도 Controller → Service → Repository 경계는 바뀌지 않습니다.

조회 구현 완료 확인

- 단건: Controller Endpoint 를 실제 호출해 존재/부재의 200/404 또는 표준 오류 Mapping, Response Body, 권한/마스킹을 확인합니다.
- 목록: Controller Endpoint 에서 0 건이 정상 빈 목록으로 반환되는지, 최대 limit·허용 sort/filter·권한/마스킹을 확인합니다.
- Offset: Controller 의 page/size 입력부터 `SamplePage(items,total,page,size)` 응답까지 확인하고 첫/마지막/빈/size 경계, deep offset, count 비용, DB3 동일 정렬을 검증합니다.
- Keyset: Controller 의 cursor/size 입력과 items/hasNext/nextCursor 응답을 확인하고 중복 정렬값, 잘못된/위변조 cursor, 데이터 추가·삭제 중 다음 페이지 중복/누락을 검증합니다.

Source: cpf-member/online/src/main/java/member/sample/controller/SampleTransactionController.java · service/SampleTransactionService.java · repository/SampleTransactionRepository.java · repository/SampleTransactionMapper.java · dto/SamplePage.java · dto/SampleSlice.java · dto/SampleSearchRequest.java · cpf-education/src/main/java/com/cpf/education/online/querypaging/controller/MemberSearchController.java · service/MemberSearchService.java · cpf-starters/data/persistence/src/main/java/com/cpf/data/persistence/api/CpfPagingAndSortingRepository.java · CpfCursorRepository.java · page/CpfPageRequest.java · page/CpfCursorCodec.java · page/CpfHmacCursorCodec.java

최소 Transaction 예

```
@CpfService
class MemberService {
    @CpfTransactional
    Member save(Member m) { return repository.save(m); }
}
```

DB3 Persistence 변경은 Oracle·PostgreSQL·MariaDB 의 Schema/Query/Migration/Runtime 영향을 같이 검증합니다. MySQL·MSSQL·H2 는 공식 Vendor 가 아닙니다.

Source: cpf-starters/web/src/main/java/com/cpf/web/api/CpfController.java · cpf-starters/data/persistence/src/main/java/com/cpf/data/persistence/api/CpfCrudRepository.java · cpf-starters/data/persistence/src/main/java/com/cpf/data/persistence/api/annotation/CpfTransactional.java

3 Transaction · Idempotency · UNKNOWN

Local DB Transaction 과 Remote Side Effect 를 구분하고, 중복·Timeout·결과 미확정 시 복구 방식을 고릅니다.

Public API 빠른 선택

표 3-1. Public API 빠른 선택

Public API 빠른 선택의 선택 기준과 확인 항목을 빠르게 비교합니다.

하려는 일	Public API / Annotation	핵심 옵션	언제 선택	실패·복구
Local DB Tx	@CpfTransactional	propagation/isolation/timeout/readOnly	한 Runtime 의 한 DB 경계	예외 시 rollback 규칙
업무 중복 방지	@CpfIdempotent	required/ttlSeconds/inProgressTimeoutSeconds/replayResult	재호출 가능한 쓰기·관리 실행	payload 충돌/진행중/UNKNOWN 을 durable store 로 판단
외부 재시도	@CpfRetry	name/maxAttempts/delayMillis/reconcileUnknownOutcome	Retry-safe 오류만	Side Effect UNKNOWN 은 blind retry 금지
Timeout	@CpfTimeout	name/timeoutMillis/fallbackMethod	외부/Remote 시간 제한	timeoutMillis=0 이면 canonical config
결과 상태	CpfResult<T>	SUCCESS/BUSINESS_FAILURE/TECHNICAL_FAILURE/UNKNOWN	Domain/Remote 결과 분기	UNKNOWN 을 실패로 덮지 않음

옵션 빠른 확인

표 3-2. 옵션 빠른 확인

옵션 빠른 확인의 선택 기준과 확인 항목을 빠르게 비교합니다.

API	옵션	필수/기본 값	언제 변경	주의
@CpfIdempotent	required	true	선택적 멍등 정책일 때만 false 검토	관리/Side Effect 는 대부분 true 유지
@CpfIdempotent	ttlSeconds	86400	중복 허용 Window 에 맞춤	업무 재처리 Window 보다 짧지 않게
@CpfIdempotent	inProgressTimeoutSeconds	300	긴 처리 시	너무 짧으면 정상 실행을 stale 로 오판

API	옵션	필수/기본 값	언제 변경	주의
@CpfIdempotent	replayResult	true	결과 재생을 금지해야 할 때	동일 요청에 일관된 결과 필요
@CpfRetry	maxAttempts	1	정말 Retry-safe 일 때 증가	1 은 retry 없음
@CpfRetry	delayMillis	0	throttling/backoff 필요 시	운영 config 와 상충하지 않게
@CpfRetry	reconcileUnknownOutcome	false	Side Effect 결과 미확정 추적 필요	true 라도 재전송 성공을 추정하지 않음

거래 패턴 선택

표 3-3. 거래 패턴 선택

거래 패턴 선택의 선택 기준과 확인 항목을 빠르게 비교합니다.

상황	권장 경계	실패 후 판단	개발자 핵심
같은 DB 안의 여러 변경	Local Transaction	Rollback/Commit	@CpfTransactional
Same JVM Domain 호출	In-process Domain Contract	호출 결과 즉시 분기	self-HTTP 금지
Remote 조희성 호출	Remote Domain/Integration	Timeout/technical failure	Retry-safe 조건 확인
Remote Side Effect	Local Tx + Remote effect 분리	UNKNOWN 가능	idempotency + probe/reconcile
다단계 장기 업무	Saga/TCC 등 정본에 정의된 패턴	단계별 보상/확정	업무 보상 규칙을 명시
XA 가능 조건	XA 선택 가능 범위만	Global outcome	운영/DB/Driver 지원조건 검증

Local Transaction 은 현재 Runtime 의 DB 경계에서 닫고, Remote Side Effect 의 결과가 확정되지 않으면 UNKNOWN 을 유지한 채 Idempotency 와 Reconcile 로 최종 상태를 확정합니다.

Source: [cpf-starters/base/runtime/src/main/java/com/cpf/reliability/api/CpfIdempotent.java](#) · [cpf-starters/integration/src/main/java/com/cpf/integration/api/annotation/CpfRetry.java](#) · [cpf-core/src/main/java/com/cpf/core/api/result/CpfResult.java](#)

실제 구현 순서

Transaction·Idempotency·UNKNOWN 은 Annotation 을 붙이는 것으로 끝내지 않고 업무 Side Effect 와 복구 경계를 함께 구현합니다.

1. Local DB 변경은 `@CpfTransactional` 경계 안에서 처리하고 외부 Side Effect 는 같은 Local Transaction 으로 원자적이라고 가정하지 않습니다.
2. 중복 가능 요청은 업무 idempotency key 를 정하고 `@CpfIdempotent` 또는 Inbox/Outbox 계약으로 재실행 결과를 확인합니다.
3. 응답 유실로 결과가 확정되지 않으면 UNKNOWN 을 유지하고 Probe/Reconcile 로 실제 결과를 확정합니다.
4. 정상·업무거절·기술실패·UNKNOWN 을 각각 Test 하고 재시도 후 중복 Side Effect 가 없는지 검증합니다.

조회 API 의 read-only Transaction 경계

단건·목록·Offset·Keyset 조회는 Controller 가 Transaction 을 직접 관리하지 않고 Service 에서 read-only Local Transaction 을 선언합니다. 실제 SQL 실행과 DB 별 Persistence 구현은 Repository 내부에 둡니다.

```
@CpfTransactional(transactionManager="cpfDomainTransactionManager", readOnly=true)
public SamplePage search(SampleSearchRequest request) {
    int page=request.safePage(), size=request.safeSize();
    List<SampleItem> items=repository.search(
        request.keyword(), request.statusCode(), page, size,
        "sample_item_id", "ASC");
    long total=repository.count(request.keyword(), request.statusCode());
    return new SamplePage(items, total, page, size);
}
```

- Controller 는 page/size/cursor 같은 HTTP 입력을 검증 가능한 DTO 로 넘기고 Response DTO 만 반환합니다. Mapper·SQL·DB Vendor 분기를 Controller 에 두지 않습니다.
- `readOnly=true`는 조회 의도를 명확히 하는 Transaction 계약이며 쓰기 방지 보안 기능으로 오해하지 않습니다. 쓰기 API 는 별도 Transaction 경계와 권한/업무 규칙을 사용합니다.
- Page 의 content/count 를 동일한 시점으로 엄격히 봐야 하는 업무는 Isolation 과 동시 변경 요구를 별도로 설계합니다. Keyset 은 stable sort 와 cursor 를 사용해 다음 구간의 중복·누락을 검증합니다.
- Repository 는 자기 Domain Owner DB 만 조회합니다. 다른 Domain 데이터가 필요하면 Repository 에서 타 DB 를 직접 조인하지 않고 공식 Domain/Public Contract 로 호출합니다.

4 Domain 호출과 외부 연계

같은 JVM·분리 WAS·외부기관 호출을 구분해 올바른 호출 계약과 Timeout/Retry 를 선택합니다.

Public API 빠른 선택

표 4-1. Public API 빠른 선택

Public API 빠른 선택의 선택 기준과 확인 항목을 빠르게 비교합니다.

하려는 일	Public API / Annotation	핵심 옵션	언제 선택	실패·복구
CPF Domain 호출	CpfDomainClient.execute(request)	typed request/result	IP/WAS 가 달라도 CPF 가 소유한 Domain Contract	CpfResult 4 상태 분기
호출 대상 선언	@CpfClient	system/operation/sideEffecting/contextRequired	typed integration/client 경계	대상·operation 추적 누락 방지
시간 제한	@CpfTimeout	name/timeoutMillis/fallbackMethod	외부/Remote latency 제한	0 은 canonical config
재시도	@CpfRetry	name/maxAttempts/delayMillis/reconcileUnknownOutcome	Retry-safe 실패	UNKNOWN blind retry 금지
거래 operation	@CpfOnlineTransaction	operationId/name/description	업무 Domain Online 거래	operationId 는 OpenAPI/Header/ADM/Log 와 동일 정보

옵션 빠른 확인

표 4-2. 옵션 빠른 확인

옵션 빠른 확인의 선택 기준과 확인 항목을 빠르게 비교합니다.

API	옵션	필수/기본 값	언제 변경	주의
@CpfClient	system	필수	대상 System 선택	임의 문자열보다 canonical systemCode 사용
@CpfClient	operation	기본 ""	대상 Operation 추적 필요	@CpfOnlineTransaction operationId 와 정합
@CpfClient	sideEffecting	false	원격 상태 변경 호출	true 이면 UNKNOWN/Idempotency 정책 필수 검토
@CpfClient	contextRequired	true	관리/비거래 예외만 검토	업무 Domain 은 true 유지
@CpfTimeout	timeoutMillis	0	Source-level 제한이 꼭 필요할 때	0 은 환경 canonical config 사용

토폴로지별 호출

표 4-3. 토폴로지별 호출

토폴로지별 호출의 선택 기준과 확인 항목을 빠르게 비교합니다.

상황	경로	Gateway	Context
Same JVM	Caller → in-process binding → Owner Domain	경유하지 않음	CpfContext 유지
분리 WAS/MSA	Caller → Remote Adapter/Registry → Owner Domain	내부 Domain 호출은 경유하지 않음	System6 serialize/deserialize
외부 Channel/Partner	External → Gateway(선택) → Target	Edge 정책이 필요하면 사용	경계에서 Context 생성/검증

Registry 와 Routing 은 Remote endpoint 를 선택하지만 업무 코드는 동일한 Domain·Operation 계약을 호출합니다. 내부 Domain 호출을 Gateway 로 우회하지 않습니다.

Canonical System6

표 4-4. Canonical System6

Canonical System6 의 선택 기준과 확인 항목을 빠르게 비교합니다.

Header	의미	불변/변경
X-Transaction-Id	거래 식별	거래 동안 유지
X-Original-System-Code	최초 발신 시스템	불변
X-System-Code	현재 시스템	hop 별 변경
X-Caller-System-Code	직전 호출자	hop 별 변경
X-Target-System-Code	대상 시스템	hop 별 변경
X-Target-Operation-Id	대상 operation	hop 별 변경

Source: cpf-core/src/main/java/com/cpf/core/api/domain/CpfDomainClient.java · cpf-starters/integration/src/main/java/com/cpf/integration/api/annotation/CpfClient.java · cpf-starters/base/runtime/src/main/java/com/cpf/foundation/execution/api/CpfOnlineTransaction.java

실제 Domain 호출 흐름

호출 대상이 Same JVM 인지 Remote 인지 업무 코드는 분기하지 않습니다. Consumer 는 Domain Operation 과 typed request/result 를 사용하고 CPF 가 Local binding 또는 Registry/Transport 를 선택합니다.

1. Service 에서 `CpfDomainClient.execute(request)`로 Domain Operation 을 호출하고 `CpfResult`의 SUCCESS/BUSINESS_FAILURE/TECHNICAL_FAILURE/UNKNOWN 을 명시적으로 분기합니다.
2. `X-Transaction-Id`, `X-Original-System-Code`, `X-System-Code`, `X-Caller-System-Code`, `X-Target-System-Code`, `X-Target-Operation-Id`는 신뢰 경계에서 생성·검증·전파합니다.
3. `@CpfTimeout`은 Remote latency 한계를 설정하고 `@CpfRetry`는 retry-safe 기술 실패에만 제한합니다. Side Effect 가 있는 UNKNOWN 은 blind retry 하지 않습니다.
4. UNKNOWN 이면 Provider 상태조회/업무 원장/Trace 로 Probe 한 뒤 결과가 확정되지 않으면 recoveryId 를 가진 Reconcile 로 넘깁니다.
5. Test 는 Same JVM 과 Remote 경로, Timeout, retry exhausted, UNKNOWN→Probe/Reconcile 을 각각 실행하고 operationId/transactionId 와 최종 업무 결과를 확인합니다.

5 오류 처리와 결과 분기

Validation·Business failure·Technical failure·UNKNOWN 을 같은 방식으로 처리하지 않고 개발자가 필요한 조치를 빠르게 고릅니다.

오류 상황별 개발자 결정

표 5-1. 오류 상황별 개발자 결정

오류 상황별 개발자 결정의 선택 기준과 확인 항목을 빠르게 비교합니다.

상황	표준 표현	재시도	개발자 조치	운영 추적
입력 검증 실패	VALIDATION_FAILED / INVALID_ARGUMENT	하지 않음	필드/업무 검증 메시지 반환	transactionId/operationId

상황	표준 표현	재시도	개발자 조치	운영 추적
대상 없음	NOT_FOUND	하지 않음	업무 의미에 맞게 404/도메인 실패	trace
동시성/중복	CONFLICT / DUPLICATE	조건부	lock/idempotency 기준 확인	audit/trace
권한 없음	UNAUTHORIZED / FORBIDDEN	하지 않음	권한/승인 경계 확인	security/audit
외부 기술 실패	EXTERNAL_SERVICE_ERROR	조건부	Retry-safe 여부 확인	operation/trace
외부 결과 미확정	EXTERNAL_UNKNOWN_OUTCOME / UNKNOWN	blind retry 금지	probe/reconcile/compensation	recoveryId/trace
DB/Infra 장애	DATABASE_ERROR / INFRASTRUCTURE_UNAVAILABLE	정책	Local rollback 후 안전 재시도 조건 확인	instance/transaction

Public API 빠른 선택

표 5-2. Public API 빠른 선택

Public API 빠른 선택의 선택 기준과 확인 항목을 빠르게 비교합니다.

하려는 일	Public API / Annotation	핵심 옵션	언제 선택	실패·복구
4 상태 분기	CpfResult<T>	fold / requireData	Domain/Remote 결과 처리	UNKNOWN 분기를 누락하지 않음
현재 Context	CpfContexts	current/requireCurrent/snapshot/capture/bind/run/call	비동기·스레드 경계 포함 추적	직접 ThreadLocal 구현 금지

결과 4 상태를 명시적으로 닫는 예

```
return result.fold(
    data -> handleSuccess(data),
    business -> handleBusiness(business),
    technical -> handleTechnical(technical),
    unknown -> enqueueReconcile(unknown));
```

금지 UNKNOWN 을 TECHNICAL_FAILURE 나 FAILED 로 바꿔 기록하거나 Side Effect 를 무조건 다시 전송하지 않습니다.

Source: cpf-core/src/main/java/com/cpf/core/api/result/CpfResult.java · cpf-core/src/main/java/com/cpf/core/api/context/CpfContexts.java

6 Messaging · Cache · File · 업무 공통

비동기·Cache·Object Storage·공통코드 같은 반복 기능을 Provider 직접 API 가 아니라 CPF Public 계약으로 사용합니다.

Public API 빠른 선택

표 6-1. Public API 빠른 선택

Public API 빠른 선택의 선택 기준과 확인 항목을 빠르게 비교합니다.

하려는 일	Public API / Annotation	핵심 옵션	언제 선택	실패·복구
메시지 소비	@CpfMessageListener	destination/consumerGroup/ idempotencyRequired/contextRequired	Broker consumer	중복·poison·DLQ/replay 고려
메시지 발행	CpfMessagingTemplate.send	CpfBrokerPublishRequest	Kafka/JMS/RabbitMQ/IBM MQ 추상화	messageId/topic/idempotencyKey
Cache	CpfCache	get/put/evict/getOrLoad/metrics/health	로컬·Redis·Valkey Provider	stampede/fail-open/serialization

하려는 일	Public API / Annotation	핵심 옵션	언제 선택	실패·복구
Object Storage	CpfObjectStorageOperations	put/get/delete/presign/multipart	첨부·대용량 파일	orphan multipart/권한/TTL
업무 공통	CpfCodeService / CpfMessageSource / CpfParameterService / CpfCalendarService	조회·메시지·파라미터·영업일	고객 업무 공통	직접 원장 접근 대신 Public Service

옵션 빠른 확인

표 6-2. 옵션 빠른 확인

옵션 빠른 확인의 선택 기준과 확인 항목을 빠르게 비교합니다.

API	옵션	필수/기본 값	언제 변경	주의
@CpfMessageListener	consumerGroup	""	공유 소비 Group 이 필요할 때	운영 topology 와 일치
@CpfMessageListener	idempotencyRequired	true	정말 중복 무해한 수신만 false 검토	기본 true 권장
@CpfMessageListener	contextRequired	true	비거래 technical listener 예외	업무 이벤트 true 유지
CpfCacheOptions	ttl	설정값	데이터 신선도에 맞춤	stale window 검토
CpfCacheOptions	negativeTtl	설정값	not-found cache 필요	실제 생성 직후 stale 주의
CpfCacheOptions	lockWait/lockLease	설정값	stampede 제어	lease 만료/장시간 loader
CpfCacheOptions	failOpen	설정값	Cache 장애에도 원본 진행 가능	보안/정합성 데이터는 신중

Source: cpf-starters/messaging/src/main/java/com/cpf/messaging/api/CpfMessageListener.java · cpf-starters/data/src/main/java/com/cpf/data/cache/api/CpfCache.java · cpf-starters/file/src/main/java/com/cpf/file/objectstorage/api/CpfObjectStorageOperations.java

실제 적용 순서

Messaging·Cache·File 은 Provider API 를 직접 흘뿌리지 않고 Public Capability 와 실패/복구 정책을 Consumer 에서 확인합니다.

5. Messaging 은 Producer/Consumer 에서 messageId-idempotencyKey·Ack/Nack·DLQ 정책을 선언하고 중복 수신 Test 를 실행합니다.
6. Cache 는 TTL·Invalidation·stale 정책과 Multi-instance refresh 를 정하고 Provider 장애 시 원본 Source fallback 또는 실패 의미를 확인합니다.
7. File/Object Storage 는 checksum·temporary upload·atomic finalize/quarantine 경계를 적용하고 중간 실패 후 잔여 파일을 확인합니다.

7 Security · 승인 · 감사 · Logging

권한이 필요한 API 와 위험 조치에 Permission/Reason/Approval/Audit 를 빠짐없이 연결합니다.

Public API 빠른 선택

표 7-1. Public API 빠른 선택

Public API 빠른 선택의 선택 기준과 확인 항목을 빠르게 비교합니다.

하려는 일	Public API / Annotation	핵심 옵션	언제 선택	실패·복구
권한 검사	@CpfPermission	value	기능/역할 권한 경계	Spring PreAuthorize expression
사전 승인	@CpfApprovalRequired	action/approvals/reasonRequired/approvalId*/reason*	위험 운영·관리 조치	승인/사유 누락 시 실행 차단
감사	@CpfAudit	action/reasonRequired/includeSafeResultSummary	중요 변경/운영 조치	민감 원문 기록 금지
업무/운영 Logging	@CpfLogging	operation/mode/includeArguments/allowlist/includeResult/ resultAllowlist/enabled	표준 구조 로그	arguments/result 기본 false

옵션 빠른 확인

표 7-2. 옵션 빠른 확인

옵션 빠른 확인의 선택 기준과 확인 항목을 빠르게 비교합니다.

API	옵션	필수/기본 값	언제 변경	주의
@CpfApprovalRequired	approvals	1	복수 승인 정책	승인자 분리/권한 확인
@CpfApprovalRequired	reasonRequired	true	사유가 불필요한 저위험 조치 예외	위험조치는 true 유지
@CpfApprovalRequired	approvalIdParameterIndex	-1	위치 기반 parameter 연동	이름 기반 속성과 충돌 주의
@CpfAudit	includeSafeResultSummary	false	마스킹된 안전 요약이 필요	원문 결과/Secret 금지
@CpfLogging	includeArguments/includeResult	false	allowlist 기반 진단 필요	PII/Secret 원문 금지

위험 조치는 Permission, Reason, Approval, Audit 가 하나의 실행 추적 안에서 이어져야 하며 민감정보는 원문으로 남기지 않습니다.

Source: cpf-starters/security/src/main/java/com/cpf/security/api/annotation/CpfPermission.java · cpf-starters/security/src/main/java/com/cpf/security/api/annotation/CpfApprovalRequired.java · cpf-starters/platform-operations/src/main/java/com/cpf/platform/operations/api/annotation/CpfAudit.java · cpf-starters/base/runtime/src/main/java/com/cpf/foundation/annotation/CpfLogging.java

보안 적용 순서

Security·승인·감사·Logging 은 Controller/Service 의 실제 위험 조치에 권한과 Evidence 를 연결합니다.

- 접근 권한은 '@CpfPermission'으로 선언하고 권한 없음 403 경로를 실제 Consumer/API Test 에서 확인합니다.
- 위험 조치는 '@CpfApprovalRequired'로 사유·승인·실행자를 연결하고 우회 실행이 불가능한지 검증합니다.
- Audit/Logging 에는 transactionId·operationId 를 남기되 PII/Secret 원문이 노출되지 않도록 Masking Test 를 수행합니다.

8 Starter · Generator · OpenAPI 연결

업무 코드가 어떤 Dependency 를 넣고 생성·OpenAPI·Frontend 까지 무엇을 함께 갱신해야 하는지 확인합니다.

Public Starter 빠른 선택

표 8-1. Public Starter 빠른 선택

Public Starter 빠른 선택의 선택 기준과 확인 항목을 빠르게 비교합니다.

영역	Artifact	언제
Profile	cpf-starter-web-api	일반 REST/Web API
Profile	cpf-starter-secure-api	인증·인가 API
Profile	cpf-starter-bff	UI/BFF
Profile	cpf-starter-event	이벤트 중심
Profile	cpf-starter-batch	Batch
Common	cpf-starter-common	코드·메시지·파라미터·영업일
Data	cpf-starter-data-jdbc / jpa / mybatis	Persistence Provider
Cache	cpf-starter-cache-caffeine / redis / valkey	Cache Provider
Messaging	cpf-starter-messaging-kafka / jms / rabbitmq / ibm-mq	Broker Provider
Integration	cpf-starter-integration-http / resilience / fixed-length	대외 연계
Security	cpf-starter-oidc / session-jdbc / session-valkey	인증/Session

- Public BOM 에 Internal Leaf 를 노출하지 않습니다.
- Framework 계약 변경은 Generator Template/Lock/Generated Domain/Sample/EDU/OpenAPI/Test 를 함께 검토합니다.
- Frontend 는 Backend OpenAPI 기반 Generated Client 와 실제 Consumer 를 연결합니다.

Generator 작업 순서

| create/setup --preview → sync/diff → compile/test → 필요 시 remove --apply

Source: cpf-docs/development/CPF_STARTER_QUICK_SELECT.md · cpf-docs/development/GENERATOR_GUIDE.md

9 검증과 완료 판단

| 코드가 컴파일되는 것만으로 끝내지 않고 변경 범위·실패·복구·DB3·Runtime 을 단계적으로 다룹니다.

검증 단계

표 9-1. 검증 단계

검증 단계의 선택 기준과 확인 항목을 빠르게 비교합니다.

단계	명령	목적	완료 판단
FAST	./gradlew cpfVerifyFast	정적/계약 빠른 확인	개발 피드백
TARGETED	./gradlew cpfVerifyTargeted -PcpfTargetCapabilities=<capability>	변경 Capability 와 횡단 영향	직접+간접 회귀
FULL LOCAL	./gradlew cpfVerifyFullLocal	Java25 + Live infra + 장애/복구 + Evidence	최종 로컬 QA

- 정상·오류·경계·부분 실패·UNKNOWN·재실행을 포함합니다.
- DB 변경은 Oracle/PostgreSQL/MariaDB Fresh→Migration→Seed→Runtime→Rollback/Recovery 까지 검증합니다.
- 미실행 검증을 PASS 로 기록하지 않습니다.

| **완료 조건** Source·Consumer·Config·SQL·Generator·OpenAPI·Frontend·Test·Evidence 가 같은 계약을 가리키고 실행 가능한 Gate 가 실제 성공해야 합니다.

10 고객 공통 Library 와 개발 환경

Generated Domain 에 공통 코드를 복사하지 않고 고객사 공통 JAR 과 개발 환경 계약을 분리해 관리합니다.

고객 공통 JAR 생성·선택 연결

`cpf library`는 고객사 공통 코드를 `customer-libraries/<name>`에 만들고 필요한 Generated Domain 에만 의존성을 연결합니다. `cpf-common`은 Framework 공통 Owner 이므로 고객사 전용 코드를 넣지 않습니다.

cpf library create → attach → sync → verify

- 공통 JAR 은 모든 Domain 에 자동 주입하지 않고 실제 사용 Domain 만 attach 합니다.
- Library 변경 시 직접 Consumer 와 Generated Domain compile/test 를 함께 확인합니다.
- Domain 을 제거하거나 재생성해도 고객사 User-owned Library Source 를 임의 삭제하지 않습니다.

Windows 경로와 Docker Lifecycle

Windows 개발 환경에서는 Repository Root 기준 파일 경로+파일명 200 자를 넘기지 않는 것을 Gate 로 사용합니다. Runtime Test 가 Docker 를 자동 기동한 경우 Health 와 기능 준비 상태를 확인하고, 정리 시 Harness 가 시작한 Container 만 종료합니다.

Source·EDU 길찾기

Public API 탐색은 Public Function TOP 100, Starter 선택은 Starter Quick Select, 생성/동기화는 Generator Guide, 실행 가능한 교육 예제는 cpf-education 의 35 개 Feature 를 기준으로 찾습니다.

Source: [cpf-tools/runtime/cli/java/CpfCli.java](#) · [cpf-docs/development/CPF_PUBLIC_FUNCTION_TOP_100.md](#) · [cpf-docs/development/CPF_STARTER_QUICK_SELECT.md](#) · [cpf-docs/development/GENERATOR_GUIDE.md](#)

개발 환경 완료 확인

로컬 환경이 “실행됨” 상태인 것과 CPF 개발 경로가 정상인 것은 다릅니다. 다음 항목을 실제 명령과 Consumer 기준으로 확인합니다.

- Repository Root 기준 경로 길이 Gate 를 통과하고 Generated/Build 산출물이 사용자 Source 경로를 침범하지 않는지 확인합니다.
- Docker 기반 Runtime 은 Container 생존이 아니라 Health·DB 연결·초기화 완료까지 확인하고 stop/reset 을 반복해도 잔여 상태가 남지 않아야 합니다.
- Public Function TOP 100 → Starter Quick Select → Generator Guide → cpf-education Sample 순서로 API 선택과 실행 예를 연결하고, 문서 예제가 실제 Source 식별자와 일치하는지 확인합니다.

로컬 개발 최종 확인

개발 환경은 도구가 실행되는 것만으로 완료하지 않고 실제 CPF 개발 경로를 한 번 끝까지 통과시켜 확인합니다.

- `cpf doctor` 또는 동등한 prerequisite 검사에서 Java·Docker·Node·Repository Root 조건을 확인하고 실패 항목을 먼저 해소합니다.
- Starter 선택 뒤 Generated Domain 을 다시 sync/build 하여 Customer Library 와 Framework Public API 가 실제 Consumer compile 경로에서 연결되는지 확인합니다.
- 로컬 Runtime 기동 후 Health 뿐 아니라 대표 API/DB 거래를 실행하고 stop/reset 후 재기동해 잔여 Container·Schema·Generated 상태가 남지 않는지 확인합니다.
- 문서의 API/옵션/예제 식별자가 Public Function TOP 100·Starter Quick Select·Generator Guide·cpf-education 현재 Source 와 일치하는지 마지막으로 대조합니다.

11 Public CLI 로 개발 수명주기 실행

일반 업무 개발자가 Framework 내부 도구를 직접 조합하지 않고 Public cpf CLI 만으로 환경 확인부터 Domain 생성·동기화·Build/Test·Runtime 확인까지 끝내는 흐름입니다.

환경 확인과 초기 준비

Public CLI 의 정보는 ``cpf-tools/runtime/cli/java/CpfCli.java``입니다. 일반 개발자는 Public Command 만 사용하고 ``dev/verify/publish/release`` Internal Namespace 를 업무 프로젝트 자동화에 직접 묶지 않습니다.

- ``cpf doctor`` — Java 25, Workspace, Domain, Capability, DB 선택과 필수 Prerequisite 를 확인합니다.
- ``cpf version`` — Framework/CLI Version, Source Identity, Capability Profile 과 Java Runtime 을 확인합니다.
- ``cpf bootstrap [--db oracle|postgresql|mariadb]`` — Public 실행환경을 준비하고 공식 DB Vendor 선택을 연결합니다.

Domain 생성·동기화·검증

- ``cpf domain-new <domain> [SYSTEM_CODE]`` — Canonical Domain 구조와 System Code 로 새 업무 Domain 을 생성합니다.
- ``cpf domain-sync [domain]`` — Generator Contract 변경을 기존 Domain 에 반영하며 Generated/User-owned 경계를 보존합니다.
- ``cpf build`` → ``cpf test`` — Compile/Test 를 수행합니다. Targeted Test 만으로 전체 영향범위 완료를 선언하지 않습니다.

개발 순서 doctor → bootstrap → domain-new → Starter/Capability 선택 → Public API 구현 → domain-sync → build/test

Runtime 기동·상태·정리

- ``cpf run`` — Public Runtime 을 기동한 뒤 Health 뿐 아니라 대표 업무 거래와 DB 결과를 확인합니다.
- ``cpf status`` — 현재 Workspace/Runtime 상태를 확인합니다.
- ``cpf stop`` — 정상 종료하고 Thread/Port/Container 등 잔여 상태를 확인합니다.
- ``cpf reset`` — 로컬 상태 초기화가 필요할 때만 영향범위를 확인하고 실행합니다.
- ``cpf help`` — 현재 Capability Profile 에 실제 노출된 Public Surface 를 확인합니다.